Overview

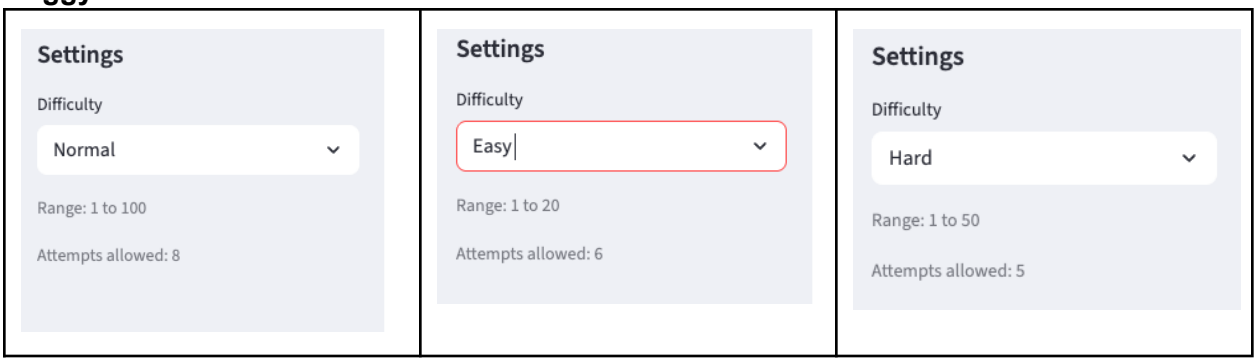
In this project, we investigate and fix bugs in the number guessing game. The goal is to reproduce incorrect behavior, identify bugs in the code, evaluate AI debugging suggestions, implement fixes, and verify that the repaired game runs correctly.

Observation 1: Difficulty ranges are inconsistent

Easy shows 1 to 20, Normal shows 1 to 100, and Hard shows 1 to 50, which makes Hard easier than Normal instead of harder.

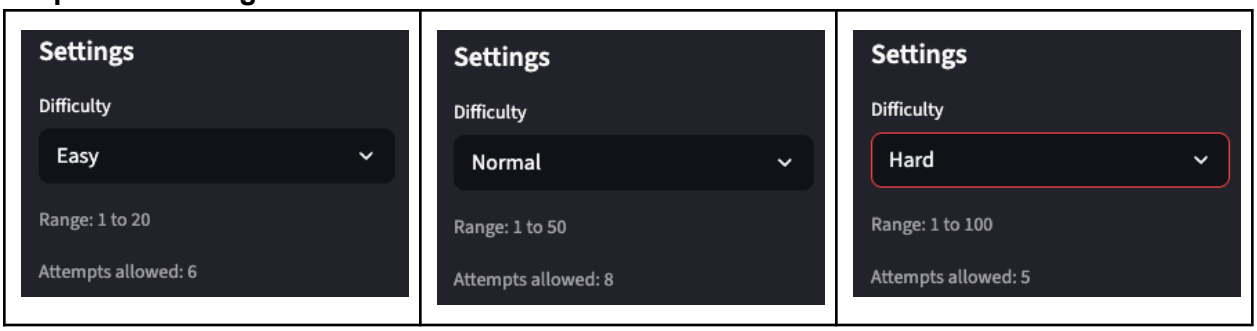
Expected: Hard mode should use a range that is at least as challenging as Normal.
Actual: Normal uses 1..100, while Hard uses 1..50. This difficulty mapping is inconsistent: Hard mode uses smaller range than normal mode, which makes hard easier than normal.

Buggy Code:

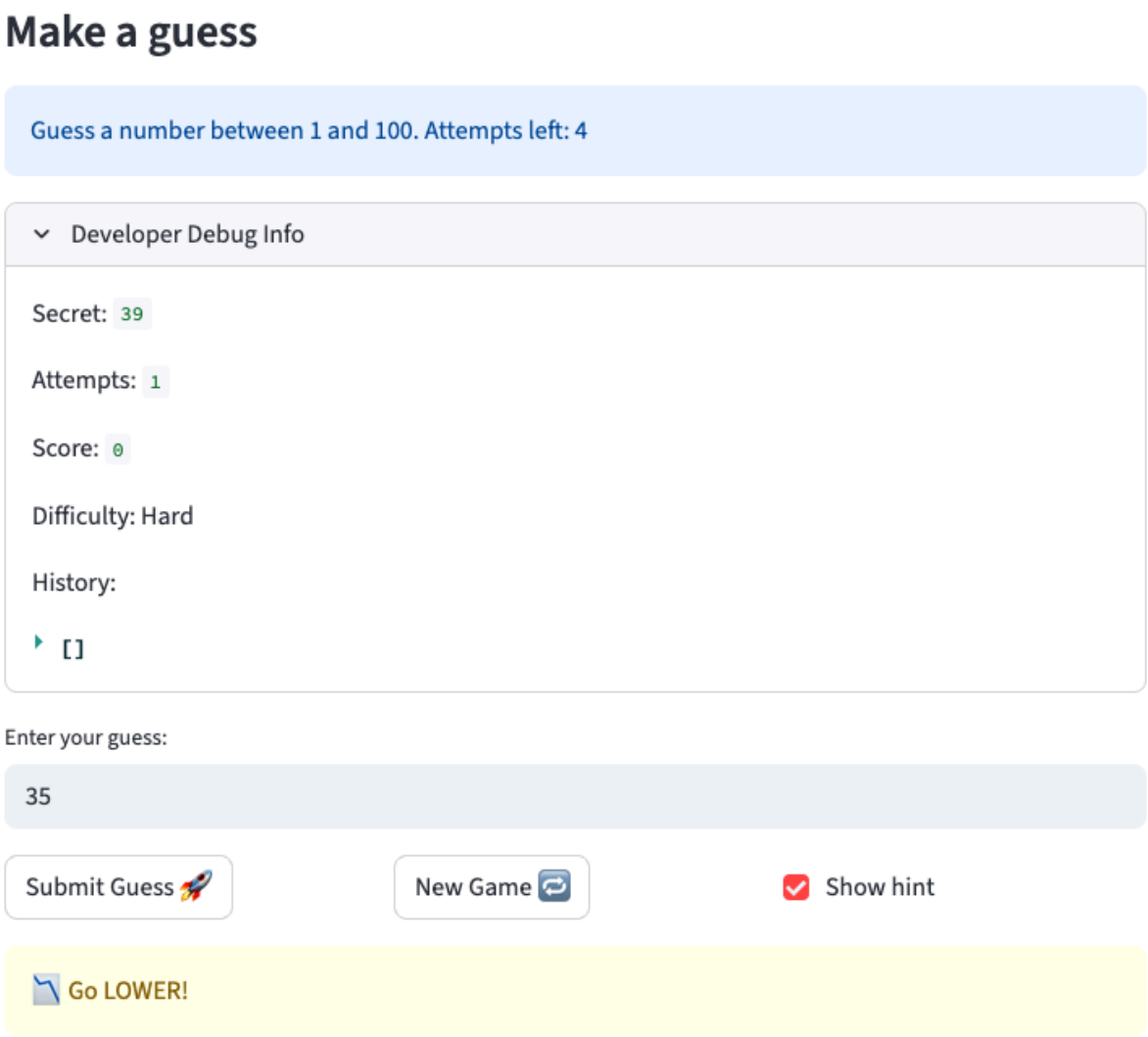


Fix in the code: The “get_range_for_difficulty()” returns (1, 100) for Normal and (1, 50) for Hard.

Output after fixing the code:



Observation 2: Hints for Higher or Lower are reversed



When I ran the game and opened Developer Debug Info, and compared my guess to the secret number. The secret is 39, the guess is 35, and the app says “Go LOWER!” even though the player should guess higher.

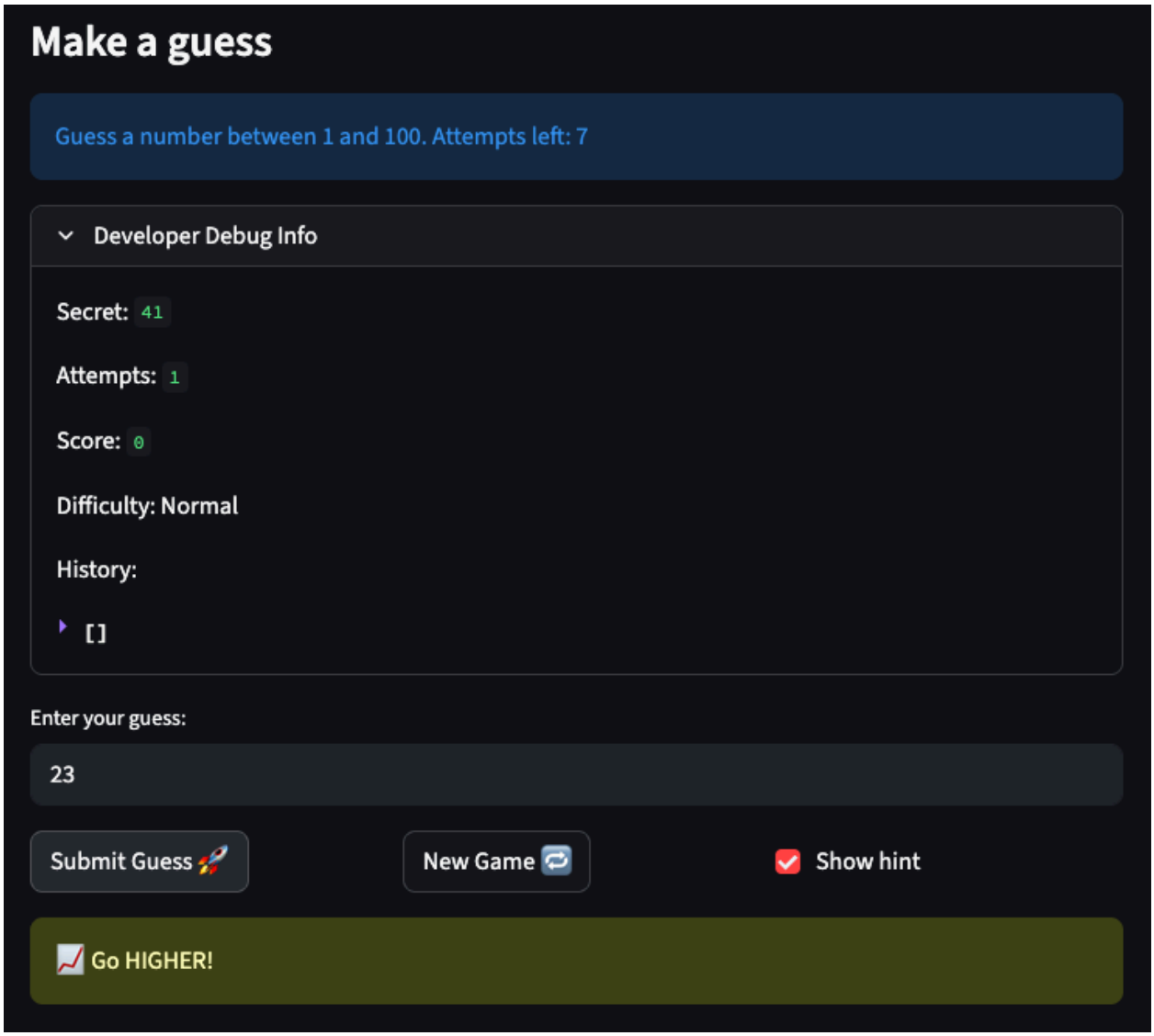
In addition, the app was converting the secret number to a string on even-numbered attempts. This is evident from lines 158-161 in “app.py” file. This is causing the “check_guess” into a “TypeError” and compares strings lexicographically instead of numbers. Here, the secret should always stay integer and guesses should always be compared numerically. What I observed was that on even attempts, comparisons can be wrong. For example, “5” > “42” is true as a string comparison, so a low guess can be reported as too high. So, check_guess(5, “42”) returns (“Too High”, “Go HIGHER!”), which is wrong for secret 42.

Expected: If the guess is below the secret, the hint should say “Go HIGHER!”.

Actual: The app says “Go LOWER!”

Bug in the code: In “check_guess()”, when guess > secret, it returns "Too High" but the message is "Go HIGHER!"; otherwise it returns "Too Low" with "Go LOWER!". Those messages are backwards.

After fixing the code:



Make a guess

Guess a number between 1 and 100. Attempts left: 6

Developer Debug Info

Secret: 41

Attempts: 2

Score: -5

Difficulty: Normal

History:

```
[
  0 : 23
]
```

Enter your guess:

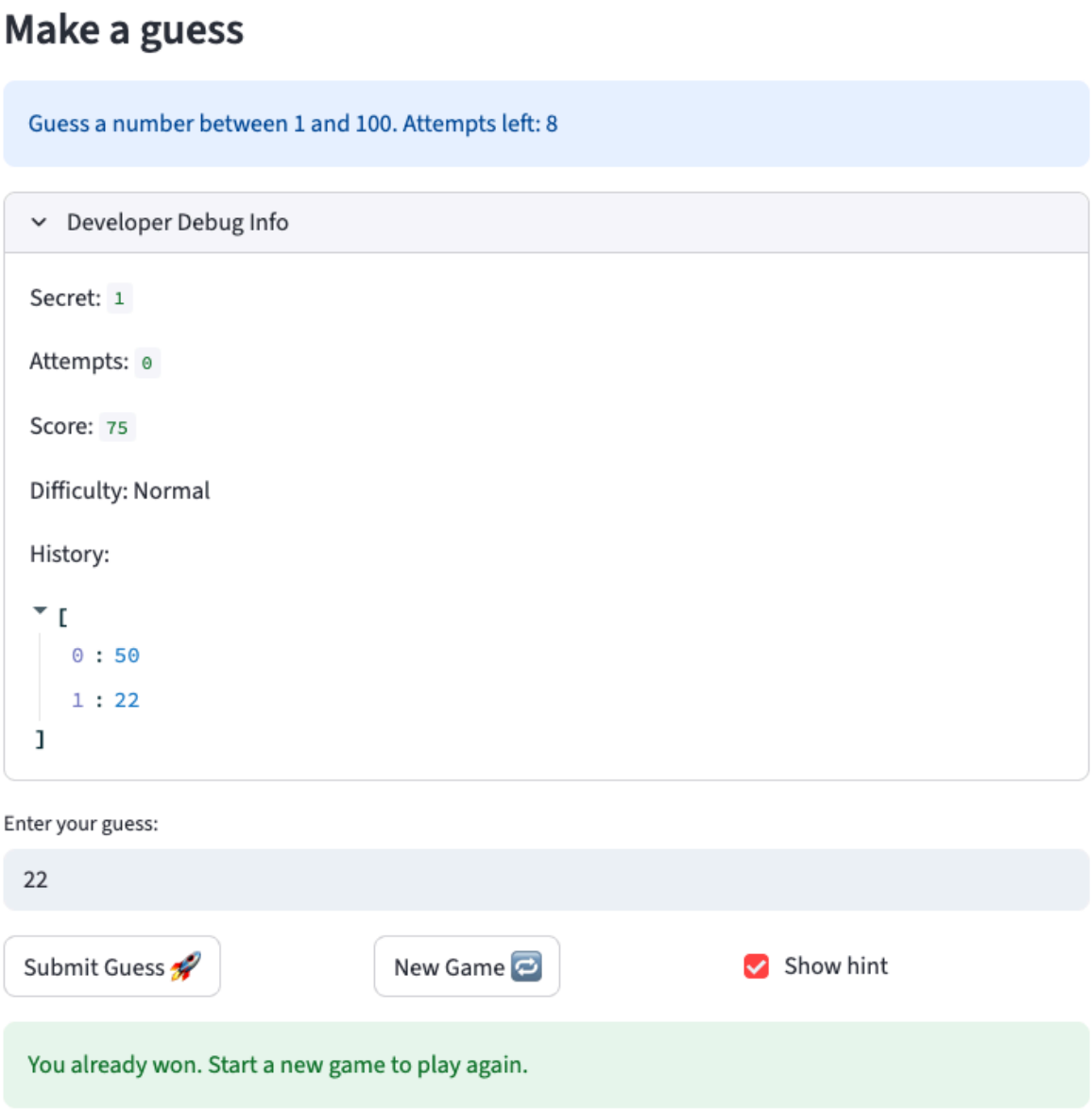
43

Submit Guess 🚀 | New Game 🔄 | Show hint

📉 Go LOWER!

Observation 3: New Game does not fully reset state

In “app.py” (lines 134-138), pressing the New Game button only resets attempts and secrets. It does not reset status, score, or history. Also, the new secret is always chosen within range (1, 100) instead of using the selected difficulty level.



Expected: A new game should reset all gameplay state and use the selected difficulty’s range.

Actual: After a win or loss, clicking New Game leaves the app in “won” or “lost” state. It also ignores the difficulty range when generating the new secret.

To reproduce this, win or lose once, click New Game, and the app can still show “You already won” or “Game over” because status was never reset.

Bug 4 (Identified using AI):

Correct AI Suggestion - Hard mode is easier than Normal

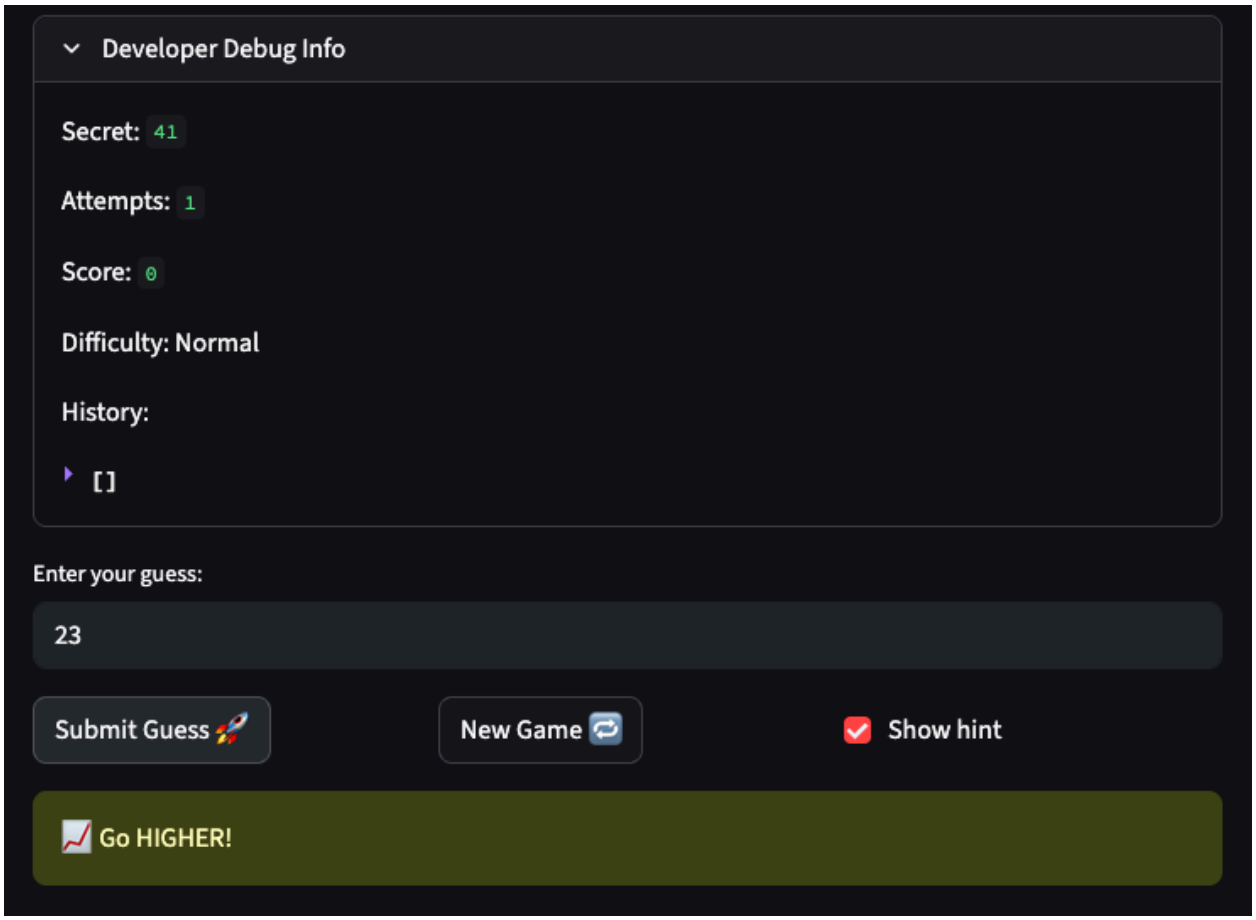
Cause: “get_range_for_difficulty()” returns (1,100) for Normal and (1,50) for Hard.

Incorrect AI Suggestion

AI suggested issues with the Streamlit app resetting the secret. Since the app already shares the secret in session state, the bigger issues were reversed hint logic and incomplete reset behavior.

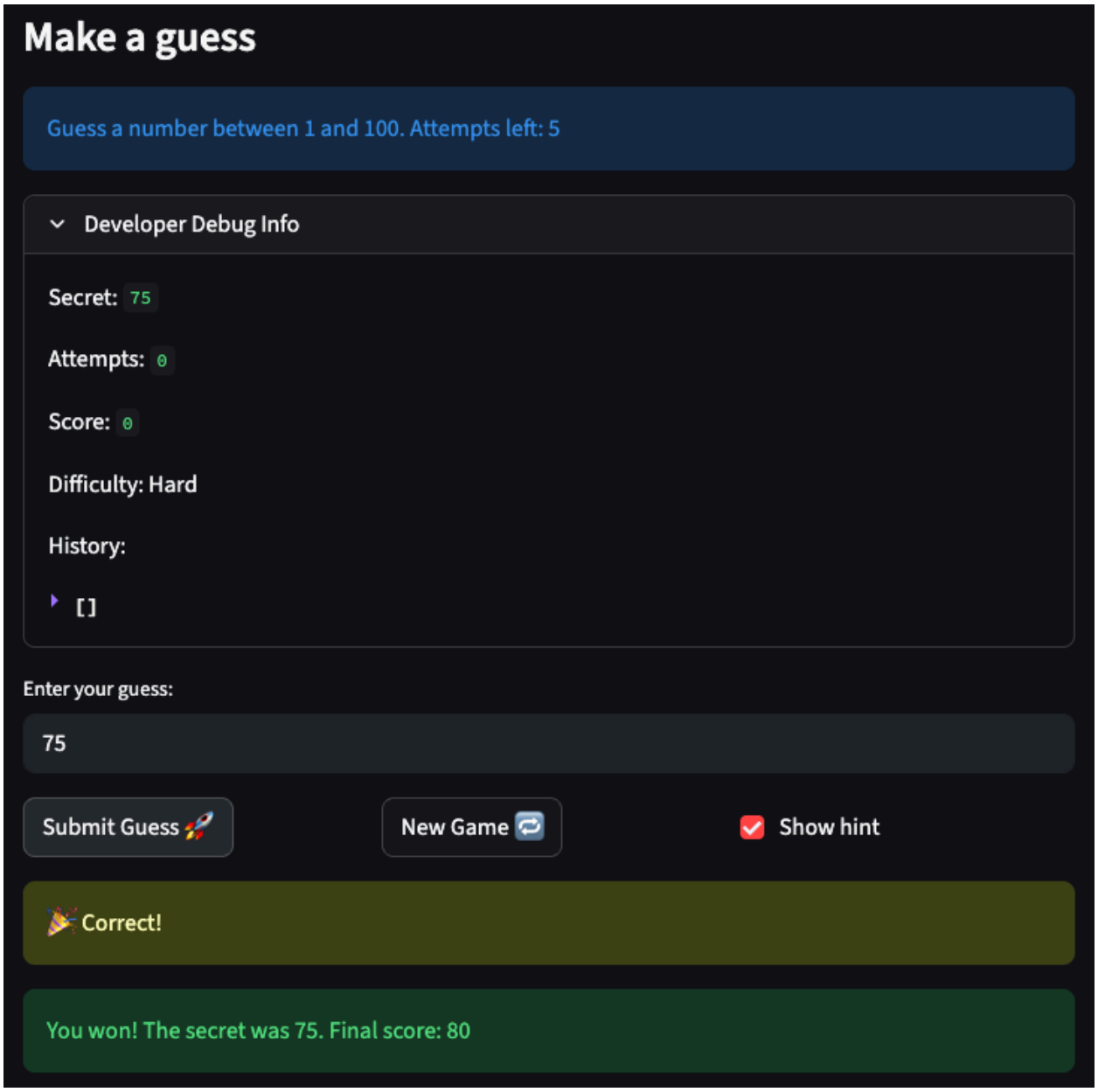
Implementation of Fixes

FIx 1: Updated “check_guess()” so “Too high” returns “Go LOWER!” and “Too Low” returns “Go HIGHER!”. The hint now matches the actual comparison result.

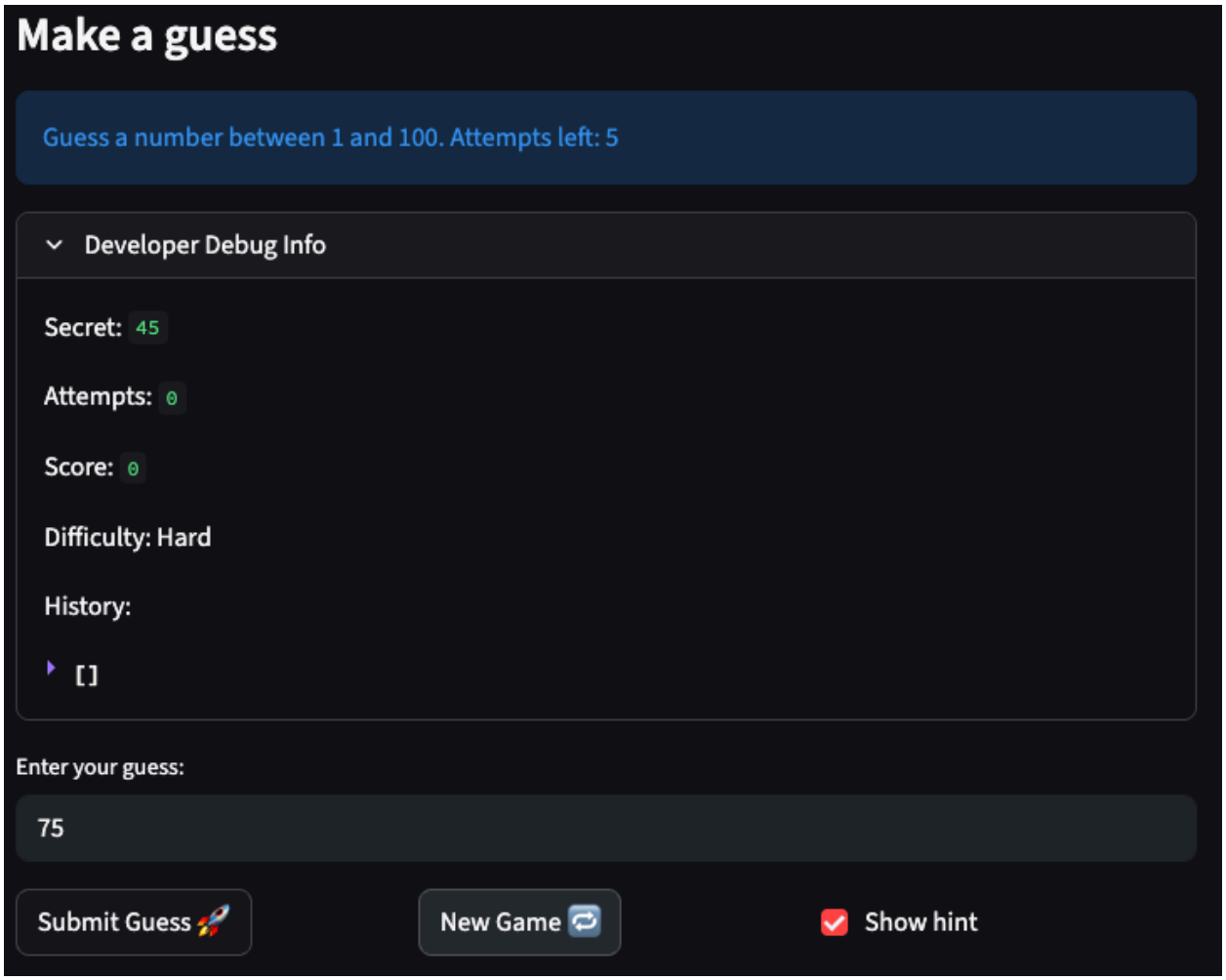


Fix 2: Updated New Game button logic to reset score, history, status, attempts, and regenerate the secret using the selected difficulty range. The game now starts from a clean state instead of showing previous game's information.

After winning the game:



The click on “New Game” resets the game board correctly



It should be noted that after fixing all the identified bugs, the game behavior matches the expectations.

Pytest Result

I ran the automated test suite using “pytest -v -s” to verify the core game logic after implementation fixes. The results show that all 11 test cases passed successfully with no failures. These tests covered the main gameplay functions, including check_guess(), get_range_for_difficulty(), and udpate_score(), confirming that the corrected logic for win detection, higher/lower hints, difficulty ranges, and score handling worked as expected.

```
(byte) jimmydani@Jimmys-MacBook-Pro ai110-module1show-gameglitchinvestigator-starter % pytest -v -s
============================================= test session starts ==============================================
platform darwin -- Python 3.10.14, pytest-9.0.2, pluggy-1.6.0 -- /opt/anaconda3/envs/byte/bin/python3.10
cachedir: .pytest_cache
rootdir: /Users/jimmydani/Documents/Pixel Playground/CodePath/AI-110/session-2/ai110-module1show-gameglitchinvesti
gator-starter
plugins: anyio-4.10.0, langsmith-0.3.45
collecting ... Running tests for check_guess...
Running tests for get_range_for_difficulty...
Running tests for update_score...
collected 11 items

tests/test_game_logic.py::test_winning_guess PASSED
tests/test_game_logic.py::test_guess_too_high PASSED
tests/test_game_logic.py::test_guess_too_low PASSED
tests/test_game_logic.py::test_easy_range PASSED
tests/test_game_logic.py::test_normal_range PASSED
tests/test_game_logic.py::test_hard_range PASSED
tests/test_game_logic.py::test_unknown_difficulty_defaults_to_normalish_range PASSED
tests/test_game_logic.py::test_update_score_win_awards_points PASSED
tests/test_game_logic.py::test_update_score_win_has_minimum_floor PASSED
tests/test_game_logic.py::test_update_score_wrong_guess_does_not_increase_score PASSED
tests/test_game_logic.py::test_update_score_unknown_outcome_leaves_score_unchanged PASSED

============================================== 11 passed in 0.01s ==============================================
```